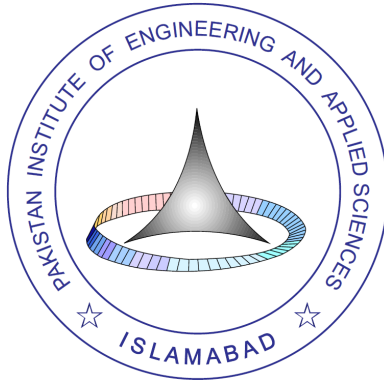




EE-414 Digital Signal Processing

Term Project

Audio Signal Processing using MATLAB

Group No.: 17

Muhammad Khuzaifa (BS-22-PB-105447)

Mian Saad Karim (BS-22-PB-103707)

Abdullah Ahmad (BS-22-PB-047631)

Submitted to: Dr. Sufi Tabassum Gul

Date of Submission: 07-05-2026

**Department of Electrical Engineering
Pakistan Institute of Engineering and Applied Sciences**

Contents

1	Abstract	1
2	Introduction	2
2.1	Objectives	2
3	Theory and Background	3
3.1	Low-Pass Filter (LPF)	3
3.2	Notch (Band-Stop) Filter	3
3.3	Real-Time 10-Band Graphic Equalizer	3
3.4	Delay-Based Voice Effects	4
4	Design and Implementation in MATLAB	5
4.1	Task 1: Filtering (AWGN + LPF + Notch)	5
4.2	Task 2: Real-Time 10-Band Graphic Equalizer	5
4.3	Task 3: Audio Effects (Echo → Flange → Reverb)	5
5	Results, Discussion and Conclusions	6
5.1	Task 1: Filtering Results (AWGN + LPF + Notch)	6
5.2	Task 2: Real-Time Equalizer Results	8
5.3	Task 3: Effects Results (Echo, Flange, Reverb)	10
	Overall Conclusion	13
	References	14
	Appendix A: Full MATLAB Code	15

Chapter 1

Abstract

This term project demonstrates three DSP tasks in MATLAB: (i) noise/interference reduction using digital filters, (ii) real-time spectral shaping using a 10-band graphic equalizer, and (iii) time-domain audio effects (echo, flange, reverb). In Task 1, a recorded voice signal is corrupted with AWGN and denoised using an LPF designed with `designfilt`, followed by notch filtering to remove a narrowband tone (approximately 1 kHz). In Task 2, real-time audio from an input device (mobile phone connected to a PC) is processed using `audioDeviceReader`, `audioDeviceWriter`, and `graphicEQ`, and verified through real-time spectrum overlays. In Task 3, echo and flange are implemented using the required delay-based equations and reverb is implemented using multiple delayed paths. The results are validated using filter responses, time-domain plots, spectrum monitoring, and listening tests via exported WAV outputs.

Introduction

Digital Signal Processing (DSP) provides mathematical and computational tools to analyze, modify, and enhance real-world signals. In audio applications, DSP is widely used for noise reduction, interference suppression, equalization, and creative effects such as echo and reverb. These techniques are fundamental in communication systems, speech processing, music production, and consumer electronics.

This project demonstrates both *offline* processing (using a recorded WAV file) and *real-time* processing (streaming audio through input/output devices). Offline processing enables repeatable experiments, precise comparisons, and saving results for evaluation, while real-time processing validates that the algorithms work continuously under frame-based constraints.

Three tasks are implemented. Task 1 focuses on filtering: adding controlled AWGN and reducing it using an LPF, and removing a single-frequency tone using a notch filter. Task 2 implements a real-time 10-band graphic equalizer to shape the spectral balance of live audio, supported by spectrum visualization. Task 3 implements delay-based effects (echo, flange, and reverb) using the required equations, with plots and listening tests to confirm expected behavior.

2.1 Objectives

1. To implement AWGN injection and denoising using an LPF designed with `designfilt` and verified using `fvtool`.
2. To design and apply a notch filter to suppress a narrowband interference tone (around 1 kHz) and validate the removal.
3. To implement a real-time 10-band graphic equalizer (32 Hz to 16 kHz) using `audioDeviceReader`, `audioDeviceWriter`, and `graphicEQ`.
4. To visualize equalizer performance using real-time spectrum overlays over approximately 30 seconds.
5. To implement echo, flange, and reverb effects using delay-line models and present results using plots and listening evaluation.

Theory and Background

This chapter provides the theoretical foundation for the digital signal processing (DSP) techniques implemented in this project. It covers the principles of frequency-domain filtering, real-time spectral modification, and delay-based audio effects.

3.1 Low-Pass Filter (LPF)

A low-pass filter (LPF) is designed to transmit frequency components below a designated cutoff frequency f_c while attenuating components above that threshold. In this project, an LPF with $f_c = 8$ kHz is utilized to denoise speech by targeting high-frequency additive white Gaussian noise (AWGN). The filter is implemented as a Finite Impulse Response (FIR) structure, which is defined by the following difference equation:

$$y[n] = \sum_{k=0}^M b_k x[n - k]$$

where M is the filter order (set to 80 in our implementation) and b_k represent the filter coefficients designed via the windowing method. FIR filters are chosen for their inherent stability and linear phase characteristics, which prevent temporal smearing of the speech signal.

3.2 Notch (Band-Stop) Filter

A notch filter is a highly selective filter that suppresses a very narrow band of frequencies around a center frequency f_0 . It is primarily used to remove single-frequency interference, such as power-line hum or tonal disturbances. The precision of the notch is dictated by the Quality Factor (Q):

$$\text{BW} = \frac{f_0}{Q}$$

By utilizing a high Q factor (set to 35 in Task 1), the bandwidth (BW) is minimized, allowing for the "nulling" of the target interference with negligible impact on the surrounding frequencies of the audio signal.

3.3 Real-Time 10-Band Graphic Equalizer

The graphic equalizer facilitates spectral shaping by dividing the audio spectrum into ten distinct bands. Each band utilizes a peaking filter centered at ISO-standard frequencies: 32, 64, 125, 250, 500, 1k, 2k, 4k, 8k, and 16k Hz.

Real-time processing is achieved through **frame-based execution**, where audio is processed in small buffers of 1024 samples. At a sampling rate of 44.1 kHz, this results in a latency of approximately 23.2 ms, ensuring that the processed output is perceptually synchronous with the input. The gains are adjustable from -20 dB to $+20$ dB, allowing for precise control over the tonal balance.

3.4 Delay-Based Voice Effects

Audio effects such as echo, flanging, and reverb rely on the manipulation of discrete-time delay lines to simulate acoustic environments or create aesthetic modulation.

1. Echo

Echo simulates a discrete reflection of sound. It is mathematically modeled as the sum of the direct signal and a delayed, attenuated version:

$$y[n] = x[n] + a \cdot x[n - d]$$

where d is the delay in samples and a is the gain coefficient. For distinct audible repetitions, d is typically set to represent a delay of 500 ms or greater.

2. Flange

Flanging is a time-varying effect created by mixing a signal with a delayed version where the delay $d[n]$ is modulated by a Low-Frequency Oscillator (LFO), usually a sine wave:

$$y[n] = x[n] + a \cdot x[n - d[n]]$$

$$d[n] = d_{base} + d_{depth} \cdot \sin(2\pi f_{LFO}n)$$

The resulting dynamic phase cancellation creates the characteristic "whooshing" comb-filter effect.

3. Reverberation

Reverberation (reverb) simulates the complex, dense reflections found in physical enclosures. Unlike a single echo, reverb is implemented using multiple parallel or series delay paths with varying gains and decay factors:

$$y[n] = x[n] + \sum_{k=1}^N a_k \cdot x[n - d_k]$$

By using at least six delayed paths, the system creates a dense thicket of reflections that results in a natural decay tail and spatial depth.

Design and Implementation in MATLAB

4.1 Task 1: Filtering (AWGN + LPF + Notch)

A 10-second voice segment is extracted from `recorded_full.wav`, converted to mono, and normalized. AWGN is added at a controlled SNR by estimating signal power and selecting appropriate noise power. An LPF is designed using `designfilt`, verified with `fvtool`, and applied using `filter`. A notch filter is then designed using `designfilt("bandstopiir")` and applied to suppress the 1 kHz interference tone. WAV outputs are exported for listening-based verification.

4.2 Task 2: Real-Time 10-Band Graphic Equalizer

Real-time audio streaming is performed using `audioDeviceReader` (input) and `audioDeviceWriter` (output). The 10-band equalizer is implemented using `graphicEQ` and the gain vector is adjusted via `equalizer.Gains`. Audio played from a mobile phone is processed in real time and sent to speakers/headphones. Spectrum overlays (input vs EQ output) are plotted during execution to provide visual verification of spectral shaping under different gain profiles.

In addition to the script-based implementation, a dedicated App Designer interface was developed for Task 2. The app provides ten vertical sliders mapped to the ISO center frequencies (32 Hz to 16 kHz), a single Start/Stop engine button, and a real-time logarithmic frequency-response plot. Internally, the app instantiates `audioDeviceReader`, `audioDeviceWriter`, and a `graphicEQ` object, and the shared callback `SliderValueChanged` updates the `Equalizer.Gains` vector directly from the slider values. The Button callback starts and stops a frame-based processing loop that continuously passes live audio through the equalizer while keeping the GUI responsive, thus providing an interactive front-end to the same real-time 10-band EQ engine described above.

4.3 Task 3: Audio Effects (Echo → Flange → Reverb)

The original 10-second segment is used as the reference signal. Echo is implemented using a delay line based on $y[n] = x[n] + a x[n - d]$. Flange is implemented using a modulated delay $d[n]$ driven by a low-frequency sine wave and mixed with the original. Reverb is implemented using multiple delayed paths with different gains (at least two) to simulate reflections and decay. The outputs are saved and plotted, and an overlay plot is included to compare all effects in a high-energy region.

Results, Discussion and Conclusions

5.1 Task 1: Filtering Results (AWGN + LPF + Notch)

Figures 5.1 and 5.2 show the LPF magnitude response with strong attenuation above the cutoff (approximately 8 kHz). Figure 5.3 shows that AWGN increases waveform fluctuations, while LPF filtering produces a smoother output. Figure 5.4 confirms a narrow attenuation band around the selected tone, and Figure 5.5 demonstrates strong reduction of the 1 kHz oscillatory component after notch filtering.

Task 1 Figures (Filtering)

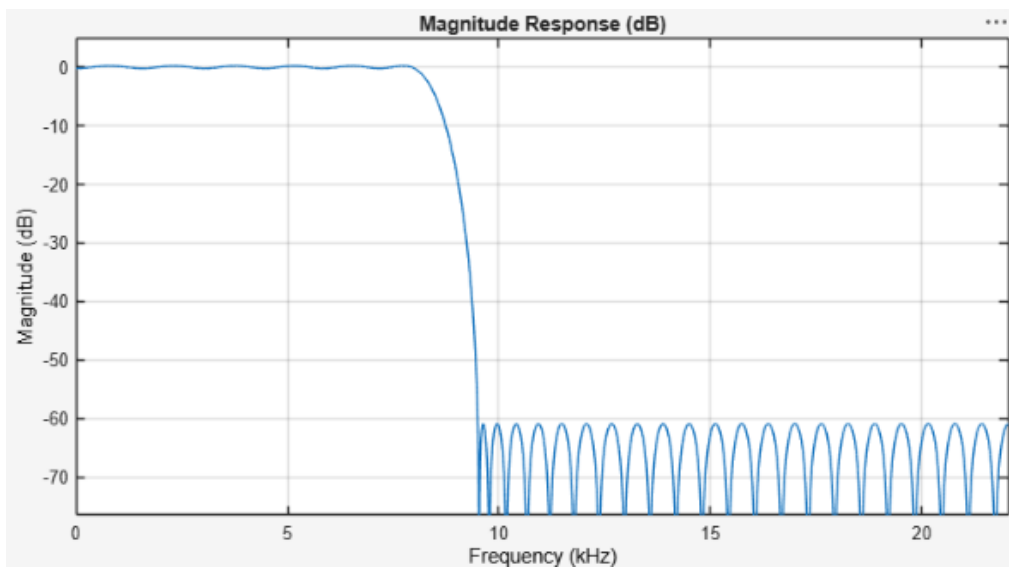


Figure 5.1: LPF magnitude response (MATLAB designfilt).

Task 1 Output Explanation

The Task 1 figures verify correct filter design and filtering performance. The LPF responses confirm strong attenuation above the cutoff, which supports AWGN reduction. The clean/noisy/filtered zoom demonstrates visible smoothing after denoising. The notch response and before/after comparison confirm selective removal of the 1 kHz tone with minimal impact on the remaining waveform.

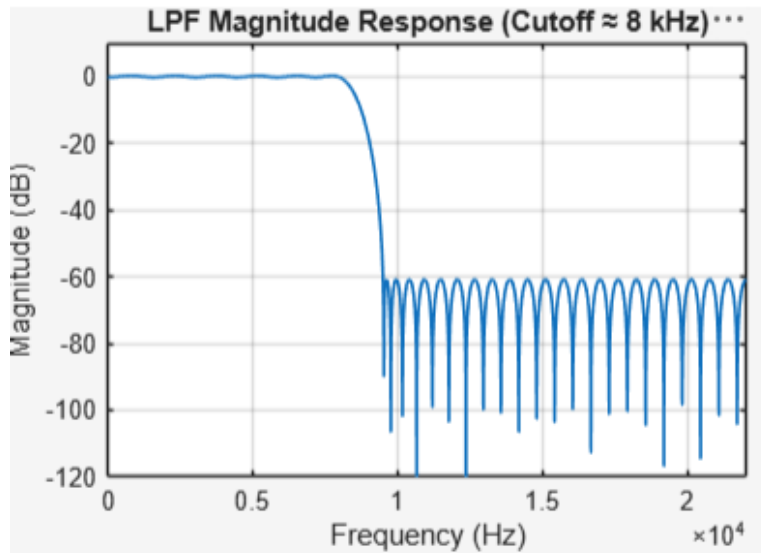
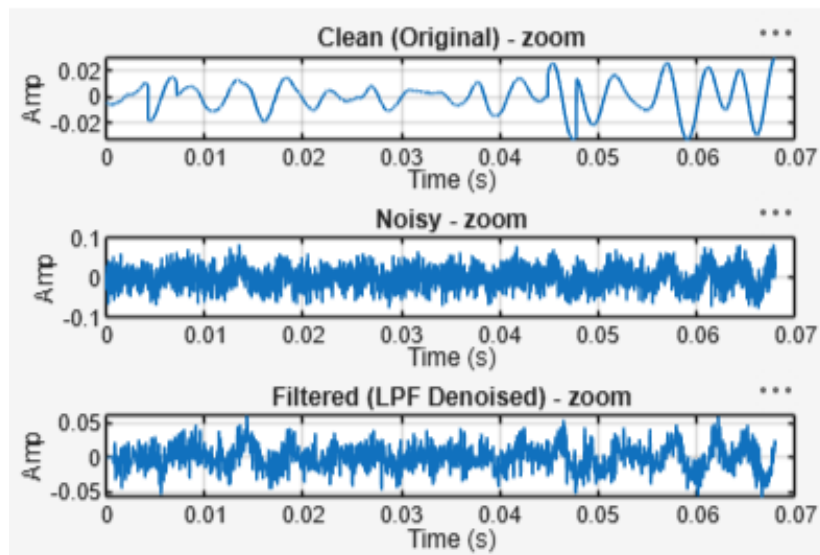
Figure 5.2: LPF magnitude response with cutoff ≈ 8 kHz.

Figure 5.3: Clean vs noisy (AWGN) vs LPF-filtered output (zoomed).

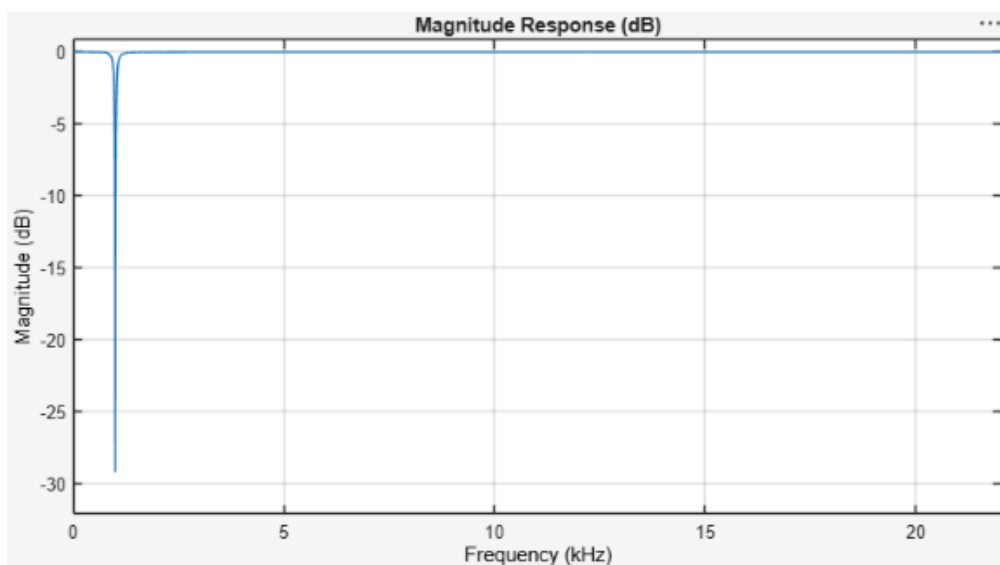


Figure 5.4: Notch (band-stop) magnitude response around the selected tone.

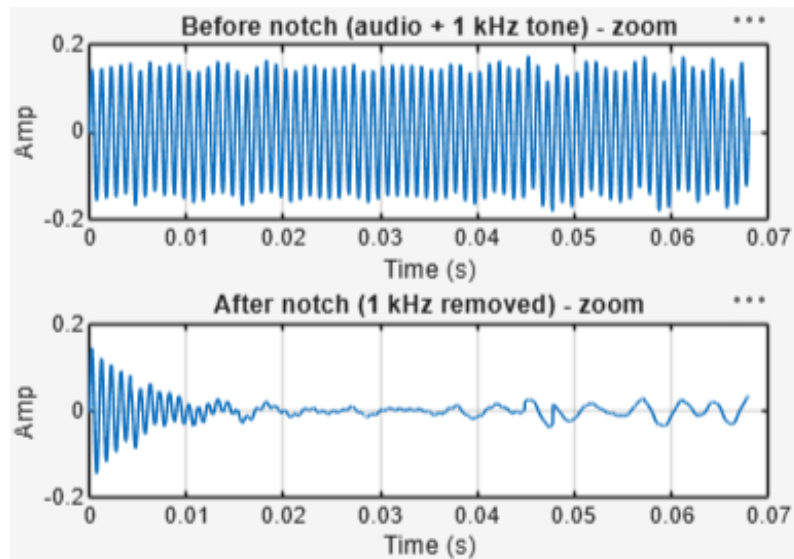


Figure 5.5: Before vs after notch filtering (1 kHz removed, zoomed).

5.2 Task 2: Real-Time Equalizer Results

Figure 5.6 provides evidence of real-time equalizer execution and gain scheduling. With flat gains (Figure 5.8), input and output spectra overlap closely. With bass-boost gains (Figure 5.9), the output magnitude increases mainly at low frequencies. With treble-boost gains (Figure 5.10), changes appear mainly at higher frequencies. These figures confirm correct band-wise control and stable real-time operation.

Task 2 Figures (Real-Time 10-Band Equalizer)

Part 02

Command Window

```
Part 2 running (10 seconds)...
Preset schedule: 0-3.3s FLAT, 3.3-6.6s BASS, 6.6-10s TREBLE
Take screenshots around 2s, 5s, 8s.
Time 3.32s: Gains(dB) -> [18 16 12 8 3 -3 -6 -6 -3 0]
Time 6.62s: Gains(dB) -> [0 -3 -6 -6 -3 3 8 12 16 18]
Saved: part2_eq_10s_output.wav
PART 2 (10s) COMPLETE ✓
```

Figure 5.6: Command Window proof of Part B execution and gain schedule.

Task 2 Output Explanation

The Task 2 figures confirm real-time processing and correct equalization behavior. The command window screenshot provides run-time evidence of execution and gain scheduling. The spectrum overlays show that flat gains preserve the spectrum, bass boost increases

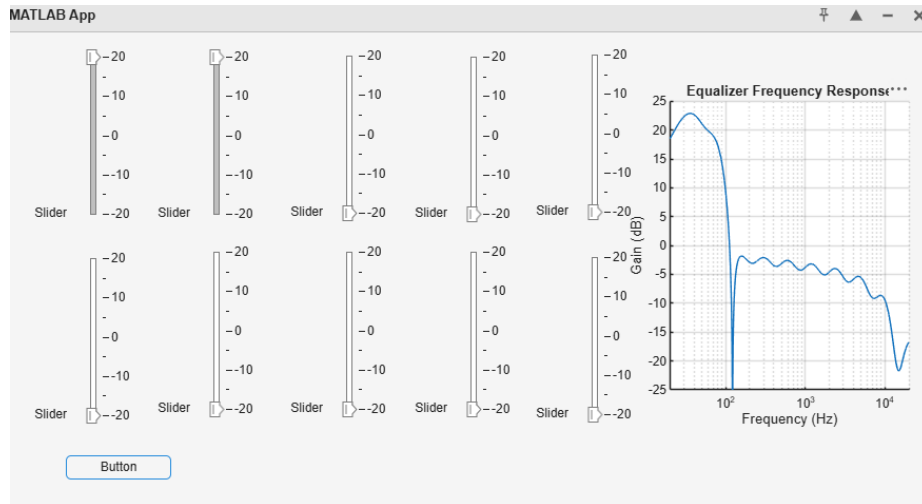


Figure 5.7: App Designer-based 10-band equalizer GUI showing sliders (32 Hz–16 kHz), Start/Stop engine button, and live logarithmic frequency-response plot driven by the `graphicEQ` object.

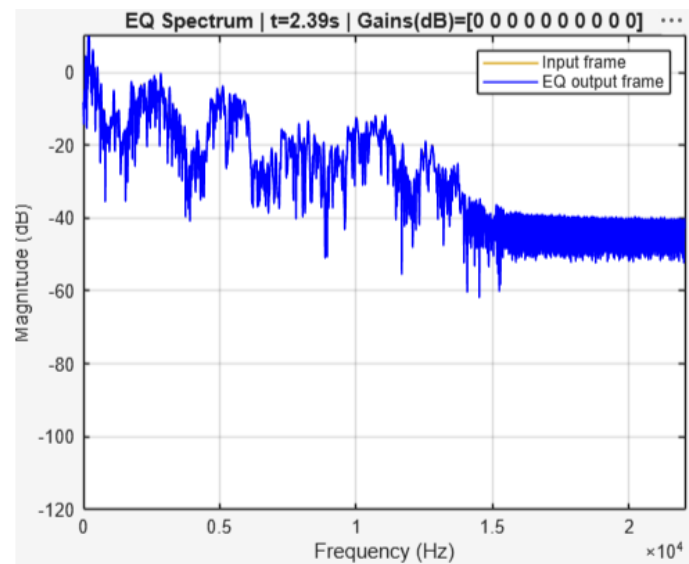


Figure 5.8: EQ spectrum overlay with flat gains (0 dB), $t \approx 2.39$ s.

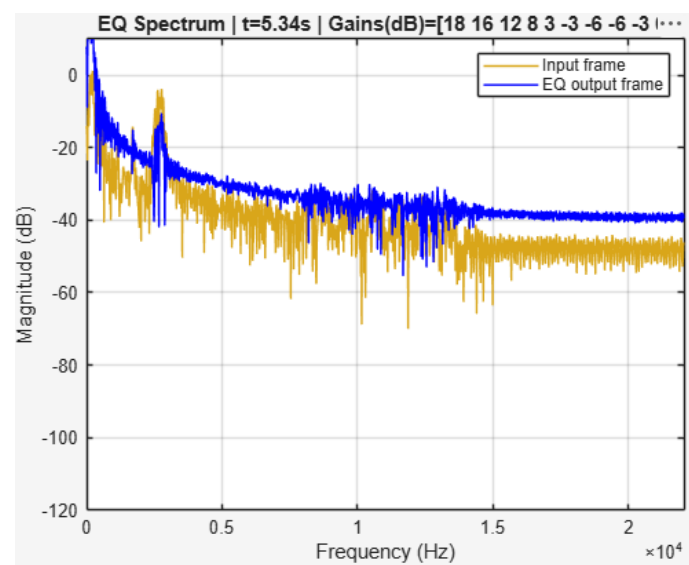


Figure 5.9: EQ spectrum overlay with bass-boost gains, $t \approx 5.34$ s.

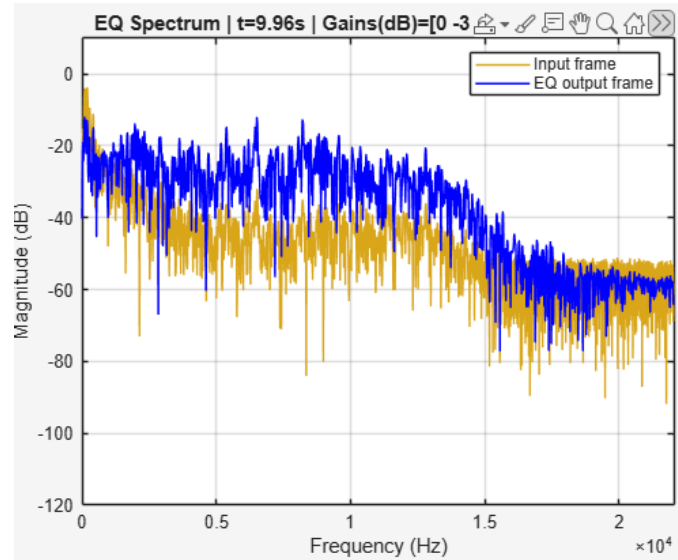


Figure 5.10: EQ spectrum overlay with treble-boost gains, $t \approx 9.96$ s.

low-frequency energy, and treble boost increases high-frequency energy, verifying correct band-wise gain application during streaming.

5.3 Task 3: Effects Results (Echo, Flange, Reverb)

Figure 5.11 shows the original waveform. Echo (Figure 5.12) introduces delayed repetitions consistent with $y[n] = x[n] + a x[n - d]$. Flange (Figure 5.13) produces time-varying modulation due to $d[n]$. Reverb (Figure 5.14) increases waveform density due to multiple delayed reflections. Figure 5.15 compares all effects in a loud region and highlights their differences.

Task 3 Figures (Original, Echo, Flange, Reverb)

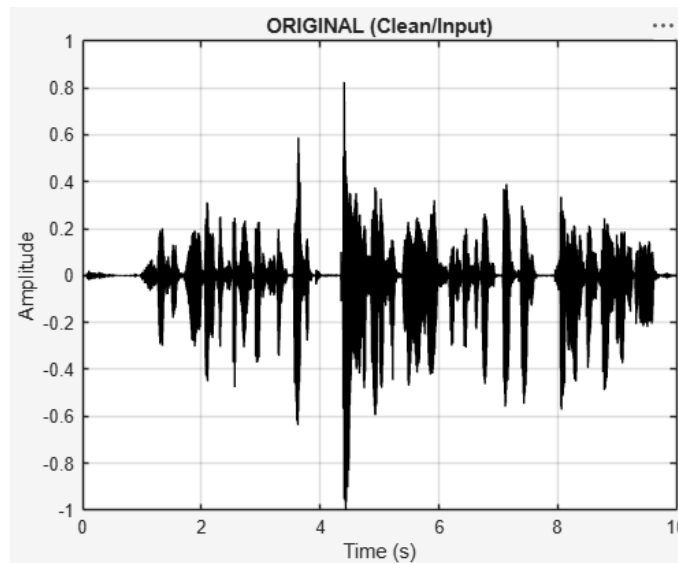


Figure 5.11: Original (clean/input) waveform (10 s segment).

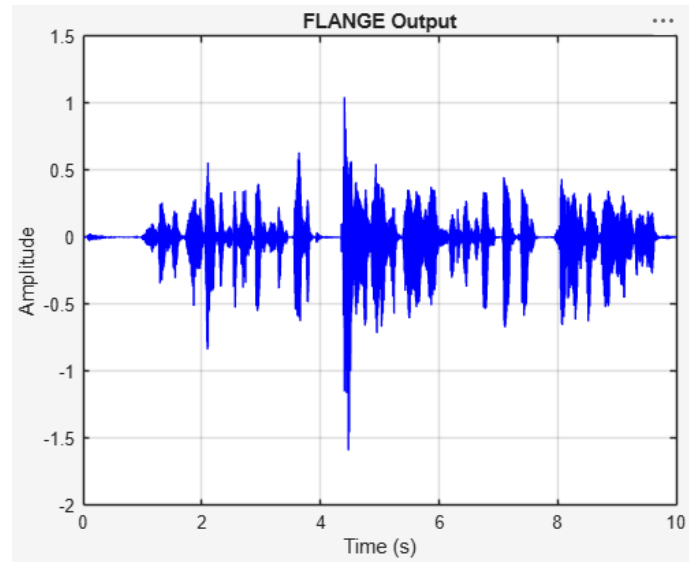


Figure 5.12: Echo output waveform (10 s segment).

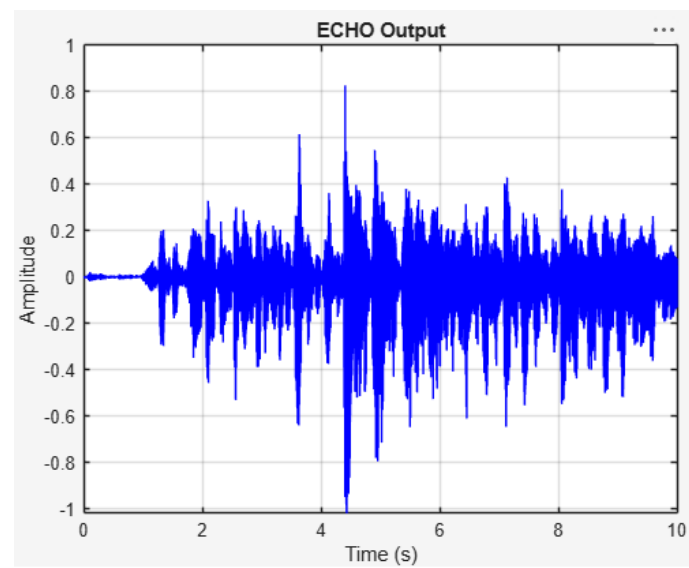


Figure 5.13: Flange output waveform (10 s segment).

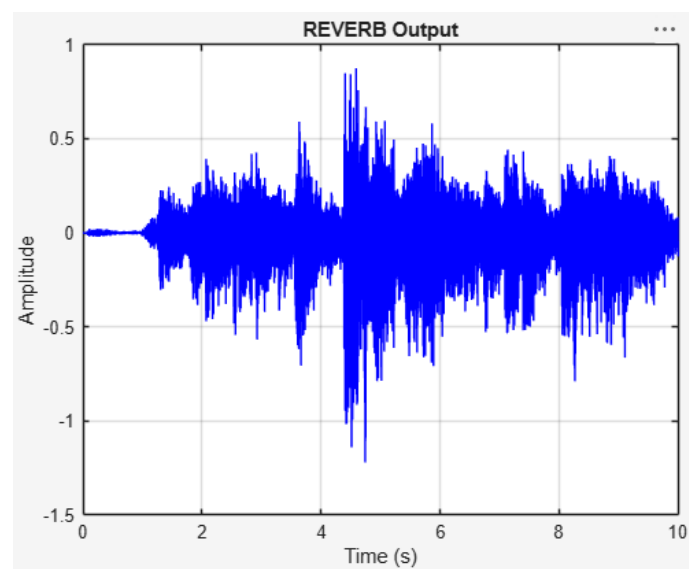


Figure 5.14: Reverb output waveform (10 s segment).

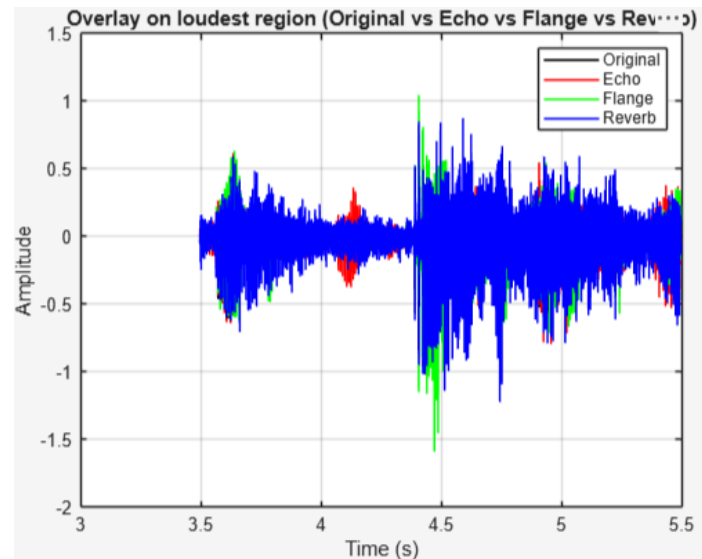


Figure 5.15: Overlay: original vs echo vs flange vs reverb (loud region).

Task 3 Output Explanation

The Task 3 figures illustrate the impact of delay-based effects. Echo introduces delayed repetitions, flange introduces time-varying modulation due to the changing delay, and reverb increases signal density through multiple reflections. The overlay plot provides a direct comparison of all effects in the same loud region, making the differences clear in both waveform shape and intensity.

Overall Conclusion

This project successfully implements and validates three practical DSP applications in MATLAB using both offline and real-time audio. The LPF responses confirm correct cut-off and attenuation, and the time-domain plots demonstrate effective reduction of broadband AWGN. The notch filter produces a narrow stopband and removes the targeted 1 kHz tone in the before/after comparison. The real-time 10-band equalizer operates stably and the spectrum overlays verify that different gain profiles reshape the output spectrum as expected. Echo, flange, and reverb effects follow the required delay-based models and produce distinct, theoretically consistent waveform and audible changes. Overall, the results satisfy the project requirements and demonstrate correct DSP design, analysis, and real-time processing workflow.

References

1. A. V. Oppenheim and R. W. Schaffer, *Discrete-Time Signal Processing*, 3rd ed., Pearson, 2010.
2. S. W. Smith, *The Scientist and Engineer's Guide to Digital Signal Processing*, California Technical Publishing, 1997.
3. MATLAB Documentation: Signal Processing Toolbox (designfilt, fvtool). <https://www.mathworks.com/help/signal/>
4. MATLAB Documentation: Audio Toolbox (audioDeviceReader, audioDeviceWriter, graphicEQ). <https://www.mathworks.com/help/audio/>
5. U. Zölzer, *DAFX: Digital Audio Effects*, 2nd ed., Wiley, 2011.

Appendix A: Full MATLAB Code

A.1 Task 1 – task1_partA_complete_v2.m

```
1 function task1_partA_complete_v2
2 clc; close all;
3
4 % --- Configuration ---
5 inFile = "recorded_full.wav";
6 segmentStartSec = 0;
7 segmentDurSec = 10;
8 lpfCutoffHz = 8000;
9 useIIR = false;
10 firOrder = 80;
11 iirOrder = 8;
12 snrDB = 10;
13 notchF0 = 1000;
14 notchQ = 35;
15 plotSamples = 2500;
16
17 % --- Audio Information ---
18 info = audioinfo(inFile);
19 disp("=== audioinfo ===");
20 disp(info);
21 Fs = info.SampleRate;
22 fileDur = info.Duration;
23
24 if fileDur < segmentStartSec + segmentDurSec
25     if fileDur <= segmentStartSec
26         error("segmentStartSec (%.2f) is beyond the file duration
27             ↪ (%.2f).", segmentStartSec, fileDur);
28     end
29     warning("File duration is %.2f s. Using segmentDurSec = %.2f s
30         ↪ instead.", fileDur, fileDur-segmentStartSec);
31     segmentDurSec = fileDur - segmentStartSec;
32 end
33
34 [x, Fs2] = audioread(inFile);
35 if Fs2 ~= Fs
36     warning("audioinfo Fs (%d) != audioread Fs (%d). Using audioread
37         ↪ Fs.", Fs, Fs2);
38     Fs = Fs2;
39 end
40
41 x = mean(x, 2);
42 x = x / max(abs(x)+1e-12);
43
44 nyq = Fs/2;
45 if lpfCutoffHz >= nyq
46     lpfCutoffHz = 0.9*nyq;
```

```

43     warning("Fs=%g Hz (Nyquist=%g). Adjusted LPF cutoff to %.0f Hz.",
44           ↪ Fs, nyq, lpfCutoffHz);
45 end
46 % --- Signal Segmentation ---
47 startSample = round(segmentStartSec*Fs) + 1;
48 endSample   = startSample + round(segmentDurSec*Fs) - 1;
49 if endSample > length(x)
50     endSample = length(x);
51 end
52 clean = x(startSample:endSample);
53
54 % --- LPF Design ---
55 if ~useIIR
56     lpf = designfilt("lowpassfir", ...
57         "FilterOrder", firOrder, ...
58         "CutoffFrequency", lpfCutoffHz, ...
59         "SampleRate", Fs);
60     lpfName = sprintf("FIR LPF (Order=%d, Cutoff=%.0f Hz)", firOrder,
61         ↪ lpfCutoffHz);
62 else
63     lpf = designfilt("lowpassiir", ...
64         "DesignMethod", "butter", ...
65         "FilterOrder", iirOrder, ...
66         "HalfPowerFrequency", lpfCutoffHz, ...
67         "SampleRate", Fs);
68     lpfName = sprintf("IIR Butter LPF (Order=%d, Cutoff=%.0f Hz)",
69         ↪ iirOrder, lpfCutoffHz);
70 end
71 fvtool(lpf); title("LPF Frequency Response");
72
73 % --- Noise Injection & Filtering ---
74 Ps = mean(clean.^2);
75 Pn = Ps / (10^(snrDB/10));
76 noisy = clean + sqrt(Pn) * randn(size(clean));
77 filtered = filter(lpf, noisy);
78
79 % --- Visualization ---
80 N = min(plotSamples, length(clean));
81 t = (0:N-1)/Fs;
82 figure("Name", "Task 1 Part A: Clean / Noisy / Filtered");
83 subplot(3,1,1);
84 plot(t, clean(1:N)); grid on;
85 title("Original (clean)"); ylabel("Amplitude");
86 subplot(3,1,2);
87 plot(t, noisy(1:N)); grid on;
88 title(sprintf("Noisy (SNR=%g dB)", snrDB)); ylabel("Amplitude");
89 subplot(3,1,3);
90 plot(t, filtered(1:N)); grid on;
91 title(sprintf("Filtered - %s", lpfName)); ylabel("Amplitude");
92 ↪ xlabel("Time (s)");
93
94 % --- Output Generation ---
95 cleanW = clean / max(abs(clean)+1e-12);
96 noisyW = noisy / max(abs(noisy)+1e-12);
97 filteredW = filtered / max(abs(filtered)+1e-12);
98 audiowrite("task1_clean_seg.wav", cleanW, Fs);
99 audiowrite("task1_noisy_seg.wav", noisyW, Fs);
100 audiowrite("task1_lpf_filtered_seg.wav", filteredW, Fs);

```

```

99 % --- Notch Filter ---
100 bw = notchF0/notchQ;
101 f1 = max(1, notchF0 - bw/2);
102 f2 = min(nyq-1, notchF0 + bw/2);
103 notch = designfilt("bandstopiir", ...
104     "FilterOrder", 2, ...
105     "HalfPowerFrequency1", f1, ...
106     "HalfPowerFrequency2", f2, ...
107     "SampleRate", Fs);
108 fvtool(notch);
109 title(sprintf("Notch Filter Frequency Response (center%.0f Hz)",
110     ↪ notchF0));
111
112 notched = filter(notch, noisy);
113 notchedW = notched / max(abs(notched)+1e-12);
114 audiowrite("task1_notch_filtered_seg.wav", notchedW, Fs);
115
116 figure("Name", "Task 1 Part A: Notch Effect");
117 plot(t, noisy(1:N), "Color", [0.85 0.65 0.1]); hold on;
118 plot(t, notched(1:N), "b"); grid on;
119 xlabel("Time (s)"); ylabel("Amplitude");
120 title(sprintf("Noisy vs Notch-Filtered (%.0f Hz)", notchF0));
121 legend("Noisy", "After Notch");
122
123 safePlay(cleanW, Fs, "CLEAN");
124 safePlay(notchedW, Fs, "NOTCH-FILTERED");
125 end
126
127 function safePlay(x, Fs, label)
128 try
129     dur = length(x)/Fs;
130     fprintf("Playing %s (%.2f s) ...\n", label, dur);
131     sound(x, Fs);
132     pause(dur + 0.25);
133 catch ME
134     warning("Playback failed for %s: %s", label, ME.message);
135 end
136 end

```

A.2 Task 2 – task2_partB_realtime_10band_eq.m

```

1 function task2_partB_realtime_10band_eq
2 clc; close all;
3
4 Fs = 44100;
5 frameSize = 1024;
6 runSec = 30;
7 bandCenters = [32 64 125 250 500 1000 2000 4000 8000 16000];
8 gainsDB = [0 0 0 0 0 0 0 0 0 0];
9 plotEveryNFrames = 3;
10 smoothing = 0.6;
11
12 try
13     d = daq.getDevices;
14     disp("=== daq.getDevices ===");
15     disp(d);
16 catch

```

```

17     disp("daq.getDevices not available on this setup (OK).");
18 end
19
20 deviceReader = audioDeviceReader( ...
21     "SampleRate", Fs, ...
22     "SamplesPerFrame", frameSize);
23 deviceWriter = audioDeviceWriter( ...
24     "SampleRate", Fs);
25
26 eq = graphicEQ( ...
27     "SampleRate", Fs, ...
28     "CenterFrequencies", bandCenters);
29
30 eq.Gains = gainsDB;
31
32 fprintf("Running real-time 10-band EQ for %g seconds...\n", runSec);
33 fprintf("Band centers (Hz): %s\n", mat2str(bandCenters));
34 fprintf("Gains (dB): %s\n", mat2str(eq.Gains));
35
36 NFFT = 4096;
37 f = (0:NFFT/2-1) * (Fs/NFFT);
38 fig = figure("Name", "TASK 2: Real-time Graphic EQ Spectrum (Input vs
    ↳ Output)");
39 ax = axes(fig);
40 hold(ax, "on"); grid(ax, "on");
41 hIn = plot(ax, f, -120*ones(size(f)), "Color", [0.90 0.65 0.10],
    ↳ "LineWidth", 1.0);
42 hOut = plot(ax, f, -120*ones(size(f)), "b", "LineWidth", 1.0);
43 xlabel(ax, "Frequency (Hz)");
44 ylabel(ax, "Magnitude (dB)");
45 title(ax, sprintf("EQ Spectrum | t=0.00s | Gains(dB)=%s",
    ↳ mat2str(eq.Gains)));
46 legend(ax, "Input frame", "EQ output frame", "Location", "northeast");
47 xlim(ax, [0 Fs/2]);
48 ylim(ax, [-120 10]);
49
50 magInPrev = -120*ones(size(f));
51 magOutPrev = -120*ones(size(f));
52
53 tic;
54 k = 0;
55 while toc < runSec && ishandle(fig)
56     x = deviceReader();
57     x = mean(x, 2);
58     y = eq(x);
59     deviceWriter(y);
60     k = k + 1;
61     if mod(k, plotEveryNFrames)==0
62         Xin = fft(x, NFFT);
63         Yin = fft(y, NFFT);
64         magIn = 20*log10(abs(Xin(1:NFFT/2)) + 1e-12);
65         magOut = 20*log10(abs(Yin(1:NFFT/2)) + 1e-12);
66
67         magInPrev = smoothing*magInPrev + (1-smoothing)*magIn;
68         magOutPrev = smoothing*magOutPrev + (1-smoothing)*magOut;
69         set(hIn, "YData", magInPrev);
70         set(hOut, "YData", magOutPrev);
71         title(ax, sprintf("EQ Spectrum | t=%.2fs | Gains(dB)=%s", toc,
    ↳ mat2str(eq.Gains)));
72         drawnow limitrate;

```

```

73     end
74 end
75
76 release(deviceReader);
77 release(deviceWriter);
78 disp("TASK 2 / PART B COMPLETE.");
79 end

```

A.3 Task 3 – task3_partC_realtime_effects.m

```

1  function part3_voice_step_by_step_play_and_plots
2  clc; close all;
3
4  inFile = "recorded_full.wav";
5  segmentStartSec = 0;
6  segmentDurSec   = 10;
7
8  echoDelayMs = 500;
9  echoMix     = 0.90;
10 echoFB      = 0.65;
11
12 flangeBaseDelayMs = 10.0;
13 flangeDepthMs     = 8.0;
14 flangeLfoHz       = 0.25;
15 flangeMix         = 0.95;
16
17 revDelaysMs = [90 140 200 280 360 450];
18 revGains    = [0.75 0.60 0.50 0.40 0.32 0.25];
19 revFB       = 0.45;
20
21 plotFull10Sec = true;
22 overlayLoudest2Sec = true;
23 overlayWindowSec = 2.0;
24
25 info = audiointro(inFile);
26 Fs = info.SampleRate;
27 [x, Fs2] = audioread(inFile);
28 if Fs2 ~= Fs, Fs = Fs2; end
29 x = mean(x, 2);
30 x = x / (max(abs(x)) + 1e-12);
31 fileDur = length(x)/Fs;
32
33 if fileDur < segmentStartSec + segmentDurSec
34     segmentDurSec = max(0, fileDur - segmentStartSec);
35 end
36
37 if segmentDurSec <= 0
38     error("Invalid segment. Check segmentStartSec and file duration.");
39 end
40
41 i0 = round(segmentStartSec*Fs) + 1;
42 i1 = min(length(x), i0 + round(segmentDurSec*Fs) - 1);
43 orig = x(i0:i1);
44 t = (0:length(orig)-1)/Fs;
45
46 fprintf("Loaded '%s' | Fs=%d | Segment %.2f{%.2f s | N=%d\n", ...

```

```

47     inFile, Fs, segmentStartSec, segmentStartSec+segmentDurSec,
        ↪ length(orig));
48
49 D = max(1, round(echoDelayMs*1e-3*Fs));
50 wetEcho = filter(1, [1 zeros(1,D-1) -echoFB], orig);
51 echoOut = (1-echoMix)*orig + echoMix*wetEcho;
52
53 N = length(orig);
54 n = (0:N-1)';
55 baseD = flangeBaseDelayMs*1e-3*Fs;
56 depthD = flangeDepthMs*1e-3*Fs;
57 d = round(baseD + depthD*sin(2*pi*flangeLfoHz*n/Fs));
58 d(d<0) = 0;
59 flangeDelaySig = zeros(N,1);
60 for ii = 1:N
61     jj = ii - d(ii);
62     if jj >= 1
63         flangeDelaySig(ii) = orig(jj);
64     end
65 end
66 flangeWet = orig + flangeDelaySig;
67 flangeOut = (1-flangeMix)*orig + flangeMix*flangeWet;
68
69 dk = round(revDelaysMs*1e-3*Fs); dk(dk<1)=1;
70 Dmax = max(dk);
71 tail = filter(1, [1 zeros(1,Dmax-1) -revFB], orig);
72 reverbOut = orig;
73 for k = 1:numel(dk)
74     Dk = dk(k);
75     reverbOut(Dk+1:end) = reverbOut(Dk+1:end) +
        ↪ revGains(k)*tail(1:end-Dk);
76 end
77
78 origW = norm_audio(orig);
79 echoW = norm_audio(echoOut);
80 flangeW = norm_audio(flangeOut);
81 reverbW = norm_audio(reverbOut);
82
83 audiowrite("part3_original_10s.wav", origW, Fs);
84 audiowrite("part3_echo_10s.wav", echoW, Fs);
85 audiowrite("part3_flange_10s.wav", flangeW, Fs);
86 audiowrite("part3_reverb_10s.wav", reverbW, Fs);
87
88 stepPlotAndPlay(orig, origW, Fs, t, "ORIGINAL (Clean/Input)", "k",
        ↪ plotFull10Sec);
89 stepPlotAndPlay(echoOut, echoW, Fs, t, "ECHO Output", "b",
        ↪ plotFull10Sec);
90 stepPlotAndPlay(flangeOut, flangeW, Fs, t, "FLANGE Output", "b",
        ↪ plotFull10Sec);
91 stepPlotAndPlay(reverbOut, reverbW, Fs, t, "REVERB Output", "b",
        ↪ plotFull10Sec);
92
93 if overlayLoudest2Sec
94     plotOverlayLoudest(orig, echoOut, flangeOut, reverbOut, Fs,
        ↪ overlayWindowSec);
95 end
96
97 disp("PART 3 COMPLETE.");
98 end
99

```

```

100 function stepPlotAndPlay(yRaw, yPlay, Fs, t, titleText, color, plotFull)
101 figure("Name", titleText);
102 if plotFull
103     plot(t, yRaw, color); grid on;
104     xlabel("Time (s)"); ylabel("Amplitude");
105     title(titleText);
106 else
107     N = min(length(yRaw), round(2*Fs));
108     plot(t(1:N), yRaw(1:N), color); grid on;
109     xlabel("Time (s)"); ylabel("Amplitude");
110     title([titleText " (first 2 seconds)"]);
111 end
112 drawnow;
113 safePlayFull(yPlay, Fs, titleText);
114 end
115
116 function safePlayFull(x, Fs, label)
117 try
118     dur = length(x)/Fs;
119     fprintf("Playing %s (%.2f s) ...\n", label, dur);
120     sound(x, Fs);
121     pause(dur + 0.25);
122 catch ME
123     warning("Playback failed for %s: %s", label, ME.message);
124 end
125 end
126
127 function y = norm_audio(x)
128 y = x ./ (max(abs(x)) + 1e-12);
129 end
130
131 function plotOverlayLoudest(orig, echoOut, flangeOut, reverbOut, Fs,
    ⇐ winSec)
132 N = length(orig);
133 win = max(32, round(0.05*Fs));
134 rmsEnv = sqrt(movmean(orig.^2, win));
135 [~, idxMax] = max(rmsEnv);
136 overlayN = min(N, round(winSec*Fs));
137 s0 = max(1, idxMax - floor(overlayN/2));
138 s1 = min(N, s0 + overlayN - 1);
139 s0 = max(1, s1 - overlayN + 1);
140 tO = ((s0:s1)-1)/Fs;
141 figure("Name", "Overlay (loudest region): Original vs Effects");
142 plot(tO, orig(s0:s1), "k", "LineWidth", 1.2); hold on;
143 plot(tO, echoOut(s0:s1), "r");
144 plot(tO, flangeOut(s0:s1), "g");
145 plot(tO, reverbOut(s0:s1), "b");
146 grid on;
147 xlabel("Time (s)"); ylabel("Amplitude");
148 title("Overlay on loudest region");
149 legend("Original", "Echo", "Flange", "Reverb");
150 end

```